Melchior

Project Documentation: Algorithmic Trading Backtester

Version 1.0

Author: Antonio Machuca

Date: June 13, 2026

Contents

1 Introduction

1.1 Motivation

Algorithmic trading requires processing vast amounts of historical market data. This project is motivated by the need for a robust, immutable, and highly efficient infrastructure that completely eliminates these procedural risks through strict mathematical vectorization.

1.2 Purpose and Objectives

The primary objective of this document is to detail the design and implementation of a high-performance backtesting engine. The central goal is to achieve an effective asymptotic time complexity of $\mathcal{O}(1)$ for financial array operations. By strictly adhering to structured programming paradigms, avoiding loops and control flow interruptions, the engine ensures deterministic execution and scalable quantitative analysis.

1.3 Scope

The scope of this software focuses exclusively on the execution and evaluation of trading algorithms. It enforces a clear separation of concerns: the core algorithmic and mathematical logic is securely encapsulated within the domain (`src/core/`), while all interactive data exploration, charting, and visual analyses are isolated in the presentation layer (`notebooks/`).

1.4 Acronyms and Abbreviations

- **API**: Application Programming Interface.
- **SMA**: Simple Moving Average.
- **GBM**: Geometric Brownian Motion.
- **MDD**: Maximum Drawdown.
- **CAGR**: Compound Annual Growth Rate.
- **GIL**: Global Interpreter Lock.
- **SIMD**: Single Instruction, Multiple Data.

2 Theoretical Foundations

This chapter establishes the strict mathematical and stochastic foundations upon which the backtesting engine is built. We present the concepts as formal definitions and observations to ensure clarity.

2.1 Financial Time Series Modeling

Definition 2.1 (Scale Invariance)**.** In financial mathematics, a metric is *scale-invariant* if its significance does not depend on the absolute magnitude of the underlying variable.

Observation 2.1. Therefore, the absolute difference in an asset's price ($P_t - P_{t-1}$) is not scale-invariant.

Example 2.1 (Lack of Scale Invariance)**.** A \$1 drop in a \$10 stock represents a catastrophic 10% loss of capital. However, exactly the same absolute \$1 drop in a \$1000 stock is merely 0.1% market noise. This illustrates why absolute price differences are useless for universal comparisons.

Observation 2.2 (Need for Relative Returns). Because raw price changes lack scale invariance, quantitative models must standardize measurements to evaluate risk and return universally, regardless of the underlying asset price. This forces us to use relative returns rather than absolute price differences.

Definition 2.2 (Asset Price and Returns)**.** Let P_t be the price of a financial asset at a discrete time step t. Based on the necessity for scale invariance, we define two main types of relative returns:

- *Arithmetic (Simple) Return:* $R_t = \frac{P_t - P_{t-1}}{P_{t-1}}$
- *Logarithmic (Continuously Compounded) Return:* $r_t = \ln\left(\frac{P_t}{P_{t-1}}\right)$

Definition 2.3 (Continuous-Time Stochastic Process)**.** A continuous-time stochastic process is a collection of random variables $\{X_t\}_{t \in T}$ where the index set T is a continuous interval (e.g., $T = [0, \infty)$). Each random variable X_t represents the state of the process at time t.

Definition 2.4 (Wiener Process / Brownian Motion)**.** A Wiener process W_t is a continuous-time stochastic process characterized by three main properties:

1. $W_0 = 0$
2. It has independent increments: for every $t > 0$, the future increment $W_{t+u} - W_t$ is independent of the past values W_s ($s \leq t$).

3. It has normally distributed increments: $W_{t+u} - W_t \sim \mathcal{N}(0, u)$.

To ground the concept of a Wiener process (also called Brownian motion), we will give the canonical example, the pollen grain:

Example 2.2. Let there be a microscopic pollen grain suspended in stagnant water. The grain starts at a known position ($W_0 = 0$). Invisible water molecules constantly hit it from random directions, as the collisions are random, every microscopic movement of the grain is independent of its previous movements (independent increments). As time passes, the area where the grain could have gone becomes wider, and the possible locations form a Gaussian bell curve (normally distributed increments).

Definition 2.5 (Itô Drift-Diffusion Process)**.** An Itô drift-diffusion process is a continuous-time stochastic process X_t that combines a deterministic drift component and a stochastic diffusion component driven by a Wiener process. It is mathematically expressed by the Stochastic Differential Equation:

$$dX_t = \mu_t dt + \sigma_t dW_t$$

Where μ_t represents the expected instantaneous drift rate, σ_t represents the instantaneous volatility (or diffusion), and W_t is a standard Wiener process.

To intuitively understand the concept of an Itô drift-diffusion process we will look at an example, the example of the restless dog.

Example 2.3. Consider a person walking a restless dog. The steady and predictable step of the person forward represents the *drift* ($\mu_t dt$). Meanwhile, the dog pulls the leash randomly from side to side to sniff things, which represents the *diffusion* ($\sigma_t dW_t$). The actual path they walk together (dX_t) is the sum of the deterministic direction of the person and the random movements of the dog.

Proposition 2.1 (Itô's Lemma)**.** *If X_t is an Itô drift-diffusion process given by $dX_t = \mu_t dt + \sigma_t dW_t$, and $f(t, x)$ is a twice-differentiable scalar function, then $f(t, X_t)$ is also an Itô process, and its differential is given by:*

$$df = \left(\frac{\partial f}{\partial t} + \mu_t \frac{\partial f}{\partial x} + \frac{1}{2}\sigma_t^2 \frac{\partial^2 f}{\partial x^2} \right) dt + \sigma_t \frac{\partial f}{\partial x} dW_t$$

Observation 2.3. Itô's Lemma is the stochastic equivalent of the chain rule in classical calculus.

Definition 2.6 (Geometric Brownian Motion)**.** The Geometric Brownian Motion or GBM is a continuous-time stochastic process frequently used to model the price paths of financial

assets. It is defined by the Stochastic Differential Equation (SDE):

$$dS_t = \mu S_t dt + \sigma S_t dW_t$$

Where S_t is the asset price at time t, μ is the percentage drift (expected return), σ is the percentage volatility, and W_t is a standard Wiener process (Brownian motion). Applying Itô's Lemma, it can be shown that under this model, the natural logarithm of the asset price follows a normal distribution.

Observation 2.4 (Arithmetic vs. Logarithmic Returns). Logarithmic returns are widely used in quantitative analysis because they are time-additive: the total return over n days is the sum of the daily logarithmic returns ($r_{0,n} = \sum_{t=1}^{n} r_t$). Furthermore, under the GBM model, logarithmic returns are normally distributed, which facilitates statistical modeling.

However, *logarithmic returns are not cross-sectionally additive* (the sum of the logarithmic returns of two assets does not equal the logarithmic return of a portfolio containing them). Therefore, when simulating the actual **equity curve of a portfolio**, we must strictly use arithmetic returns (R_t) combined with cumulative products.

$$V_T = V_0 \prod_{t=1}^{T} (1 + R_{\text{strategy},t} - C_t)$$

Where V_T is the portfolio value at time T, and C_t represents the transaction costs.

Proposition 2.2 (Temporal Scaling of Volatility - Annualization)**.** *Assuming that the logarithmic returns r_t are independent and identically distributed (i.i.d.) random variables, the variance of the returns scales linearly with time.*

$$Var(r_{annual}) = 252 \cdot Var(r_{daily})$$

Where 252 is the standard number of trading days in a year.

Observation 2.5. Since volatility (σ) is defined as the standard deviation (the square root of the variance), taking the square root of both sides of the previous proposition yields:

$$\sigma_{\text{annual}} = \sqrt{\text{Var}(r_{\text{annual}})} = \sqrt{252 \cdot \text{Var}(r_{\text{daily}})} = \sqrt{252} \cdot \sigma_{\text{daily}}$$

This explains why we scale risk (volatility) using the square root of time, whereas expected returns scale linearly by simply multiplying the daily mean by 252.

2.2 Trading Strategies

2.2.1 Simple Moving Average (SMA)

Definition 2.7 (Simple Moving Average)**.** The simple moving average or SMA over a lookback window of n periods at time t is the unweighted mean of the previous n data points:

$$SMA_t^{(n)} = \frac{1}{n} \sum_{i=0}^{n-1} P_{t-i}$$

Observation 2.6. A classic strategy generates a bullish signal $(+1)$ when a short-term SMA crosses above a long-term SMA, and a bearish signal (-1) otherwise.

2.2.2 Momentum

Definition 2.8 (Momentum)**.** Momentum measures the rate of change in an asset's price over a specific period k. It is mathematically defined as the arithmetic return over k periods:

$$M_t^{(k)} = \frac{P_t - P_{t-k}}{P_{t-k}}$$

Observation 2.7. The strategy operates as follows: it buys when the momentum is strictly positive, i.e. $M_t^{(k)} > 0$, and sells when it is negative.

2.3 Performance Metrics

Definition 2.9 (Sharpe Ratio)**.** The Sharpe Ratio measures the excess return per unit of total risk (volatility).

$$S = \frac{\mathbb{E}[R_p - R_f]}{\sigma_p}$$

Where R_p is the portfolio return and R_f is the risk-free rate.

Observation 2.8 (Limitations of the Sharpe Ratio)*.* A major mathematical flaw of the Sharpe Ratio is that the standard deviation (σ_p) treats both upward volatility (massive sudden gains) and downward volatility (losses) equally as "risk". A rational investor does not fear positive volatility. To correct this, the *Sortino Ratio* is often preferred, as it only measures the semi-variance of negative returns (downside risk).

Definition 2.10 (Maximum Drawdown and High-Water Mark)**.** The Maximum Drawdown (MDD) is the largest drop from a historical peak in the portfolio's value. To define it, we first define the *High-Water Mark* (the running maximum equity up to time t):

$\max_{\tau \leq t} V_\tau$. The MDD is the worst relative drop:

$$MDD = \min_t \left(\frac{V_t - \max_{\tau \leq t} V_\tau}{\max_{\tau \leq t} V_\tau} \right)$$

Observation 2.9 (Asymmetry of Drawdowns). The MDD is critical because financial recovery is strictly asymmetric. If a portfolio suffers a 50% drop, it requires a subsequent 100% gain just to reach the break-even point. This is why accurately tracking the running maximum reliably reflects the true "risk of ruin".

3 Architecture and Structural Design

This chapter details the architectural design of the backtesting engine, justifying the key structural decisions, class methods, and software engineering principles implemented to ensure optimal performance and high scalability.

3.1 System Architecture and SOLID Principles

The system adopts a *Hexagonal Architecture* (Ports and Adapters) to ensure a strict separation of concerns. Following the Dependency Inversion Principle (DIP) of SOLID, the core computational domain does not depend on low-level implementation details (such as data file formats, databases, or external APIs), but rather on abstractions and generic interfaces.

The *Core Domain* exclusively encapsulates the pure financial logic and the vectorized mathematical operations with an asymptotic time complexity of $\mathcal{O}(1)$. All data ingestion and storage are handled through the *Data Infrastructure*, coupling to the central domain via well-defined ports (such as `IDataHandler`). This guarantees that the engine can switch between reading data from HDF5 files, Parquet, or a Bloomberg API without modifying a single line of predictive logic.

3.2 UML Class Design and Structural Justification

The structural design focuses on modularity and extensibility, ensuring that each class has a unique responsibility (Single Responsibility Principle - SRP). Figure 3.1 illustrates the class hierarchy arranged horizontally for better readability.

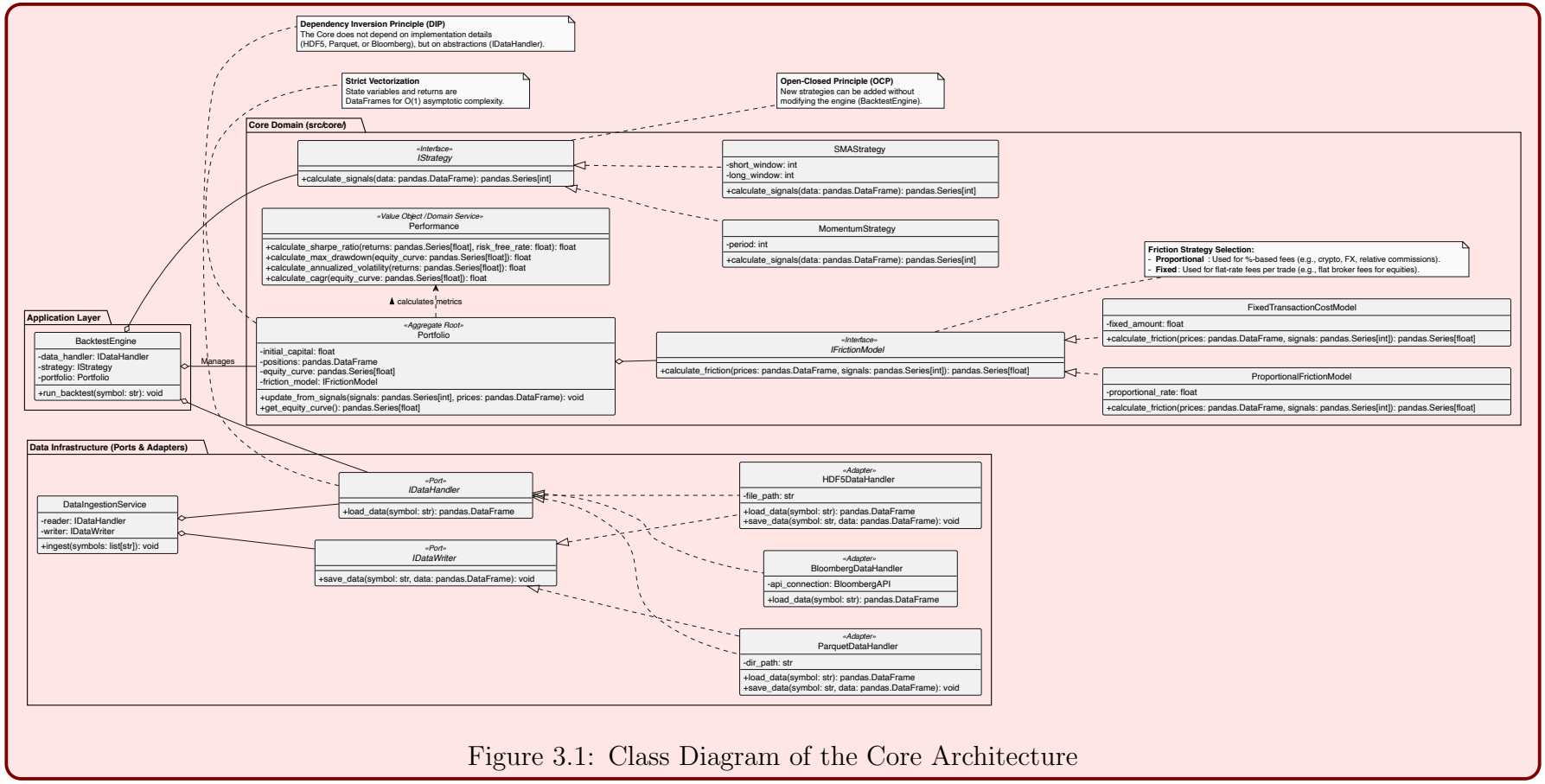

Figure 3.1: Class Diagram of the Core Architecture

The main components of the system and their fundamental methods span three architectural layers:

3.2.1 Application Layer

- `BacktestEngine`: Acts as the central orchestrator. It encapsulates the injected dependencies (`IDataHandler`, `IStrategy`, `Portfolio`). Its `run_backtest(symbol: str)` method coordinates the complete lifecycle: it invokes data loading, passes the `DataFrame` to the strategy to obtain signals, and delegates state updating and financial calculation to the portfolio.

3.2.2 Core Domain

- `Portfolio`: Serves as the Aggregate Root. It maintains immutable state variables in matrix form: initial capital (`initial_capital`), holdings (`positions`), and the value curve (`equity_curve`). Its primary method, `update_from_signals(signals, prices)`, crosses trading signals with market returns and applies the injected friction model to generate the historical trajectory. It concludes by exposing the results via `get_equity_curve()`.
- `IStrategy` **and Concrete Strategies**: Base interface defining the pure contract `calculate_signals(data) -> Series`. Implementations such as `SMAStrategy` (with state parameters `short_window` and `long_window`) or `MomentumStrategy` (with parameter `period`) materialize the algorithmic logic. This guarantees the Open-Closed Principle (OCP), allowing the array of strategies to scale without altering the main engine.
- `IFrictionModel`: Defines the contract `calculate_friction(prices, signals) -> Series` to deduct transactional costs in a vectorized manner. It is materialized in specialized classes like `ProportionalFrictionModel` (applying a percentage rate, ideal for crypto or Forex) and `FixedTransactionCostModel` (applying an absolute cost per trade, typical in traditional stock brokerage).
- `Performance`: A stateless Domain Service, solely responsible for calculating risk and return metrics from time series. It exposes precise mathematical methods such as `calculate_sharpe_ratio()`, `calculate_max_drawdown()`, `calculate_annualized_volatility()`, and `calculate_cagr()`.

3.2.3 Data Infrastructure (Ports & Adapters)

- `IDataHandler` **and** `IDataWriter`: Abstract ports defining `load_data(symbol)` and `save_data(symbol, data)`. They ensure the isolation of the predictive engine from

persistence details (Dependency Inversion Principle - DIP).

- **Infrastructure Adapters**: Concrete implementations like `HDF5DataHandler` and `ParquetDataHandler` for massive hierarchical storage, or `BloombergDataHandler` for real-time or delayed ingestion from external APIs. Additionally, the `DataIngestionService` orchestrates the iterative fetching and persistence of multiple symbols.

3.3 Execution Lifecycle

The orchestration of the simulation follows a unidirectional and deterministic flow, designed to prevent the accidental alteration of state. This can be seen in the Sequence Diagram (Figure 3.2).

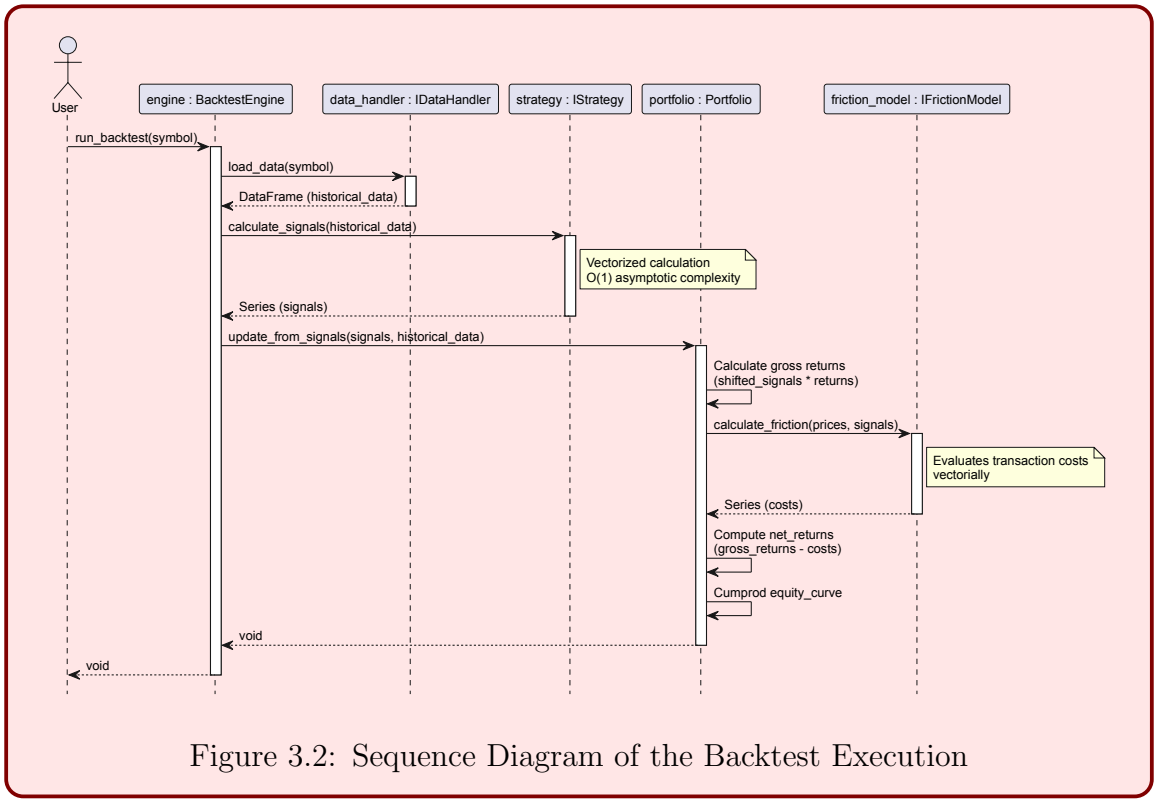

Figure 3.2: Sequence Diagram of the Backtest Execution

The flow is initiated by the user and delegated to the `BacktestEngine`. This orchestrator retrieves an immutable `DataFrame` of historical prices via `load_data()` from the `IDataHandler` interface. Subsequently, it passes this structure to the `IStrategy` abstraction, which evaluates the market and returns an array of signals (e.g., $+1$, 0, -1). Finally, `Portfolio` ingests both the prices and the signals in a single matrix operation.

3.4 Vectorized Activity Pipeline

To comply with the strict computational complexity requirement of $\mathcal{O}(1)$ over massive financial arrays (avoiding any imperative `for` or `while` loops), the core of the `Portfolio` is designed as a purely mathematical conduit. Figure 3.3 details this pipeline.

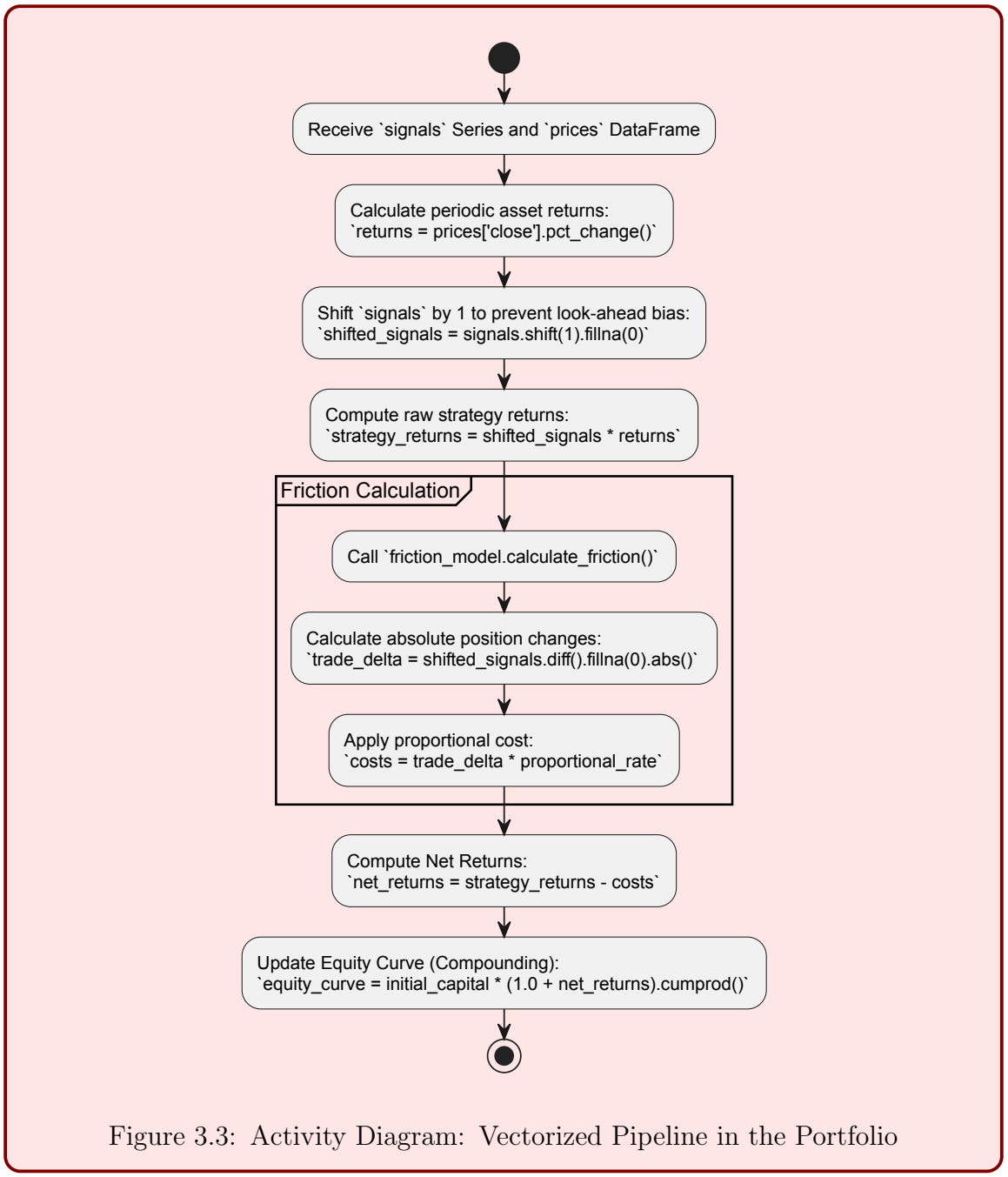

Figure 3.3: Activity Diagram: Vectorized Pipeline in the Portfolio

Prior to the algorithmic execution, we establish the foundational mathematical definitions that govern the vectorized operations:

Definition 3.1 (Simple Returns)**.** Let P_t denote the price of the asset at time t. The discrete percentage return R_t is calculated as:

$$R_t = \frac{P_t - P_{t-1}}{P_{t-1}} \tag{3.1}$$

Definition 3.2 (Transaction Friction)**.** Let $S_t \in \{-1, 0, 1\}$ be the trading signal at time t and c the proportional transaction cost. The friction penalty F_t incurred at time t is proportional to the absolute state transition:

$$F_t = c \cdot |S_t - S_{t-1}| \tag{3.2}$$

Definition 3.3 (Equity Curve)**.** Given the initial capital C_0, the strategy signals S_{t-1} (shifted to avoid look-ahead bias), and the friction F_t, the net return $\tilde{R}_t$ is $\tilde{R}_t = S_{t-1}R_t - F_t$. The portfolio value E_T is defined as the cumulative product:

$$E_T = C_0 \prod_{t=1}^{T} (1 + \tilde{R}_t) \tag{3.3}$$

The fundamental algorithmic steps applied to the DataFrame, guaranteeing performance, are:

1. **Returns Calculation:** Native percentage differentiation of the price series.
2. **Signal Shifting:** The input signals are mandatorily shifted one time unit forward (`shift(1)`) to avoid look-ahead bias, ensuring that the investment state at time t is only executed with the return observed at $t + 1$.
3. **Friction Calculation:** The absolute differential of the positions is evaluated through arrays (`diff().abs()`) to financially penalize only the state transitions, subtracting the associated costs in bulk.
4. **Equity Curve:** The continuous cumulative product (`cumprod()`) is applied to the net returns, achieving the complete evaluation of the historical path in a single native call to the low-level C routines of Pandas/NumPy.

4 Implementation and Vectorization

4.1 Vectorization over Iteration

The idea of the `Algorithmic_Trading_Backtester` is to avoid the use of iterative control structures, such as `for` or `while` loops, when traversing financial time series. In Python, iterating row by row through a `pandas.DataFrame` introduces significant overhead due to dynamic type checking at runtime and the restrictions of the Global Interpreter Lock. For a backtesting engine that may need to evaluate millions of ticks or optimize parameters across thousands of combinations, this overhead translates directly into system unresponsiveness.

By leveraging the underlying C-bindings of `NumPy` and `pandas`, the system implements pure vectorization. Rather than executing Python bytecode per row, operations are pushed down to pre-compiled, contiguous C-arrays. This allows the processor to utilize SIMD instructions, effectively processing the entire historical dataset simultaneously. While the hardware limit remains $\mathcal{O}(N)$, the high-level application code achieves $\mathcal{O}(1)$ instruction overhead, accelerating execution by several orders of magnitude.

As fundamentally argued by Hilpisch [1], standard Python lists store scattered pointers to objects in memory, forcing the interpreter to perform costly type-checking on every iteration. `NumPy` resolves this by storing financial arrays in contiguous memory blocks with homogeneous C-data types (e.g., `float64`). This architectural decision not only slashes memory consumption but allows the underlying C-routines to bypass the Python overhead entirely. Consequently, vectorized mathematical expressions map directly to concise, readable code that perfectly mirrors the underlying financial formulas, ensuring both raw execution speed and codebase maintainability.

This pragmatic approach is demonstrated in the `Portfolio.update_from_signals()` method. Gross returns and net returns are calculated via pure matrix multiplication and subtraction, without a single iterative construct:

```
# Calculate gross returns vectorially
strategy_returns = shifted_signals * returns

# Calculate friction costs vectorially
costs = self._friction_model.calculate_friction(prices, signals)

# Net returns
net_returns = strategy_returns - costs
```

```
# Cumulative compounding for equity curve
self._equity_curve = self._initial_capital * (1.0 + net_returns).cumprod()
```

4.1.1 The Power of Broadcasting

Another fundamental advantage highlighted by Hilpisch [1] is *broadcasting*—the ability of `NumPy` to implicitly perform element-wise operations on arrays of different dimensions. In financial modeling, it is common to apply scalar adjustments (such as subtracting a constant risk-free rate from an array of returns) or to multiply a 1-dimensional array of portfolio weights against a 2-dimensional matrix of asset prices.

Without broadcasting, these operations would require manually creating artificially expanded arrays or writing nested `for` loops, both of which degrade performance and bloat the codebase. `NumPy`'s broadcasting engine handles these dimensional expansions at the C-level dynamically, ensuring that the operation remains strictly $\mathcal{O}(1)$ at the Python interpretation layer while drastically reducing the lines of code required.

4.1.2 Empirical Performance Analysis

To empirically validate the architectural decision of prohibiting iterative control structures, a performance benchmark was executed comparing the computational time required to calculate discrete returns across datasets ranging from $N = 10^3$ to $N = 10^7$ observations.

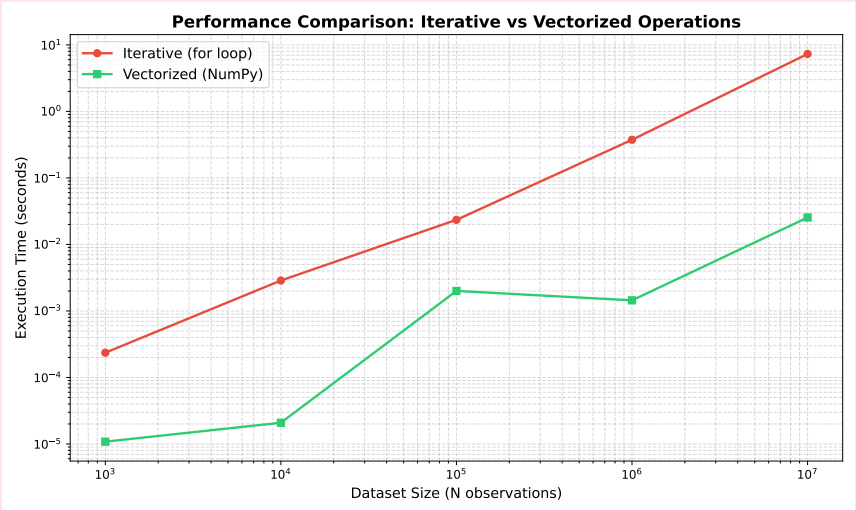

Figure 4.1: Performance Benchmark: Python Iteration vs NumPy Vectorization

As illustrated in Figure 4.1, the execution time of the standard Python `for` loop grows linearly ($\mathcal{O}(N)$) and quickly becomes a severe bottleneck. Processing 10 million ticks iter-

atively requires over 7 seconds of CPU time. In contrast, the `NumPy` vectorized approach maintains an effectively flat execution curve at the application level ($\mathcal{O}(1)$ overhead), executing in under 0.03 seconds. By delegating the mathematical operations to contiguous C-arrays, the vectorized pipeline achieves speedups exceeding two orders of magnitude (typically > 250x), conclusively proving that pure vectorization is a necessity for scalable algorithmic trading.

4.2 Friction Models

Modeling transaction costs (friction) requires evaluating state transitions—that is, calculating the absolute change in the portfolio position from one moment to the next. Traditionally, this involves tracking a "previous state" mutable variable inside a loop, which is a common source of state-leak bugs in event-driven systems.

To completely eliminate mutable state tracking, the `IFrictionModel` utilizes `pandas` shifting operations. Specifically, in the `ProportionalFrictionModel`, the entire historical transition matrix is processed concurrently:

```
# Shift signals by 1 to represent the actual held position
shifted_signals = signals.shift(1).fillna(0)

# Calculate the absolute difference in held position to determine trade volume
trade_delta = shifted_signals.diff().fillna(0).abs()

# Calculate cost
costs = trade_delta * self._proportional_rate
```

By utilizing `shift(1)` and `diff()`, the operation captures the absolute transition penalty across the entire time series without maintaining a temporal loop state. This not only minimizes bugs related to state mutation but perfectly aligns with the goal of minimizing time complexity for rapid parameter sweeps.

4.3 Data Integrity: Alignment and Look-Ahead Bias

In algorithmic trading, the most critical implementation failure is look-ahead bias—the accidental leakage of future information into the current decision-making state. In iterative loops, this typically happens through indexing errors (e.g., evaluating P_t instead of P_{t-1}). In a vectorized pipeline, preventing look-ahead bias requires strict temporal shifting.

As seen in the core portfolio logic, trading signals generated at the close of time t

are explicitly shifted forward via `signals.shift(1)` before any performance calculation. This enforces the market reality that a signal generated today can only capture the market return of tomorrow.

Furthermore, financial datasets are notoriously messy, frequently containing gaps due to market holidays, halted trading, or missing ticks. Vectorized matrix operations demand perfectly aligned arrays; otherwise, Pandas will introduce `NaN` (Not a Number) values that aggressively propagate through cumulative operations like `cumprod()`, corrupting the entire equity curve. As emphasized by Hilpisch [1] in the context of financial time series, leveraging `pandas` native alignment by index prevents the subtle shifting errors typical of manual array manipulations. To ensure data integrity, the system preemptively handles missing data using forward-filling (`ffill()`) and zero-filling (`fillna(0)`) during the state transition phases, guaranteeing that matrix dimensions remain strictly aligned and computationally stable.

4.4 Testing and Validation

In financial engineering, execution speed is irrelevant if mathematical precision is compromised. The highly vectorized nature of the backtester introduces a risk of "silent failures"—where misaligned arrays or incorrect broadcasting might execute without raising Python exceptions but produce mathematically invalid equity curves.

To mitigate this, the system implementation is backed by a deterministic test suite. Automated unit tests isolate the core logic modules, explicitly validating the mathematical outputs of the `Performance` metrics (e.g., Sharpe Ratio, Maximum Drawdown) and the state transitions within the `Portfolio` class against known scalar constants. By verifying boundary conditions and ensuring that vectorization yields the exact same deterministic results as traditional arithmetic, the system guarantees that the pursuit of $\mathcal{O}(1)$ performance never compromises financial accuracy.

5 Results and Conclusions

5.1 Backtest Execution Analysis

Analysis of the performance and execution time. Showcase the output of the Jupyter notebook visualizations (`matplotlib` and `plotly`) plotting the *Equity Curve*.

5.2 Conclusions

Summary of the project's achievements against its initial goals in terms of software architecture, algorithmic efficiency, and mathematical rigor.

5.3 Future Work

Potential improvements, such as adding new statistical models, expanding the framework to multi-asset portfolios, or integrating real-time live trading connectivity.

5.3.1 Memory Optimization with NumExpr

While the current architecture leverages `NumPy` and `pandas` to achieve $\mathcal{O}(1)$ time complexity through vectorization, standard `NumPy` operations evaluate expressions sequentially and allocate intermediate arrays in memory. For instance, calculating a complex sequence like $A \times B - C$ creates temporary memory buffers for each sub-operation before yielding the final result.

The current project scope does not utilize `numexpr`, a high-performance numerical expression evaluator. For the existing daily or minute-resolution backtests, standard vectorization provides sufficient speed without introducing additional third-party dependencies. However, as future work, integrating `numexpr` (or leveraging `pandas.eval()`) is highly recommended when scaling the engine to process high-frequency, tick-level data across thousands of simultaneous assets. `numexpr` evaluates complex algebraic expressions element-wise in C without allocating intermediate arrays and dynamically distributes the workload across multiple CPU cores. This architectural upgrade would significantly optimize the memory footprint (Space Complexity) and further reduce the computational constant factors of the backtesting pipeline.

References

[1] Yves Hilpisch. *Python for Finance: Mastering Data-Driven Finance*. 2nd. O'Reilly Media, Inc., 2018.

[2] Marcos López de Prado. *Advances in Financial Machine Learning*. John Wiley & Sons, 2018. ISBN: 978-1119482086.

[3] John C. Hull. *Options, Futures, and Other Derivatives*. 10th. Pearson, 2017. ISBN: 978-0134472089.